

AI-Based Storage Tier Recommendation for Medical Image Repositories using Deep Learning Analytics

Research Gap Analysis and Literature Survey (Papers 1 to 5)

Course Instructor: Dr. Priya V | Submission: 30 July 2026

Name: Pratyush Chandrasekhar

Reg No: 24BIT0226

1. What This Project Is About

Hospitals store a very large number of scans such as CT, MRI and X-ray. Keeping all of them on fast storage is expensive. Moving all of them to cheap storage is also a problem, because cheap storage can take hours to give a file back, and a doctor cannot wait that long.

Amazon S3 offers different storage classes. S3 Standard is fast but costly. Glacier Deep Archive is very cheap but slow. The aim of this project is to use deep learning to decide which storage class each scan should go into.

For this, I studied five research papers. For each paper I have written what the authors did, what data they used, what is good about it, what is missing, and what gap is left that our project can fill.

2. The Five Papers I Studied

#	Title	Journal	Year
P1	Cloud storage tier optimization through storage object classification	Computing (Springer)	2024
P2	Recent Advances in Big Medical Image Data Analysis Through Deep Learning and Cloud Computing	Electronics (MDPI)	2024
P3	Using Compressed JPEG and JPEG2000 Medical Images in Deep Learning: A Review	Applied Sciences (MDPI)	2024
P4	Deep learning based medical image compression using cross attention learning and wavelet transform	Scientific Reports (Nature)	2025
P5	A Recommendation System Based on AI for Storing Block Data in the Electronic Health Repository	Frontiers in Public Health	2022

3. Research Gap Analysis (Paper by Paper)

3.1 Paper 1 — Cloud storage tier optimization through storage object classification

A. Q. Khan, M. Matskin, R. Prodan, C. Bussler, D. Roman and A. Soylu, *Computing*, vol. 106, pp. 3389-3418, 2024.

In one line: This paper sorts cloud files into four storage tiers using simple rules, but it never looks inside the file.

What the authors did: The authors sort stored files into four tiers: premium, hot, cold and archive. Each file is given a score based on three things , its size, its age, and how often it is opened. If the score crosses a fixed limit, the file goes into that tier. For example, a score above 0.7 means the file goes to

the hot tier. They also try a second method using game theory, where the score weights adjust themselves instead of being fixed by hand.

Data they used: They did not have real company data. So they built their own test data from a public access log on Kaggle. It covers 871 days, 14,321 files and about 2.9 million read requests. File sizes were missing from the log, so they added random sizes and made four versions, from 1 TB up to 67 TB of total data. They used Google Cloud prices to calculate the cost.

Advantages:

- Uses four tiers instead of the usual two. This matches Amazon S3 well, because S3 also has four main classes.
- The method is simple and fast to run. It does not need a big training system, so it can be run again and again on many files.
- It works on any cloud platform and in any industry. The authors mention healthcare as one possible area.
- The game theory version gave a slightly lower cost and also ran faster.

Limitations:

- It was tested only on made-up data. The authors themselves say that testing on real data is still pending.
- The cost calculation leaves out the fee charged for moving a file between tiers, and the penalty for deleting a file too early. So the real savings will be lower than what they report.
- It only saves money if files are kept for about 90 days or more.
- The score limits, such as 0.7, were chosen by hand. If the usage pattern changes, someone has to set them again manually.
- It only reads file details. It never opens the file or looks at what is inside it.

RESEARCH GAP

This method only looks at file details such as size, age and how many times the file was opened. It never looks inside the image. In a hospital this is a problem. Two scans can have the same size, the same age and the same number of views, but one may show a normal result and the other may show a problem that the doctor is still following up. This method would treat both scans in exactly the same way. So the important scan may get sent to archive storage, where it takes hours to bring back. The paper also has no way to keep a file in storage for the number of years that hospital rules require.

How we can improve on this: We can add one more input to the score — a value that comes from looking at the image itself using a deep learning model. Along with this we can also use DICOM details like the scan type and the body part. The four-tier idea is good and we will keep it, but instead of setting the limits by hand we will learn them from the hospital's own record of which scans were opened again. We will also add the S3 transfer charges that this paper left out.

3.2 Paper 2 — Recent Advances in Big Medical Image Data Analysis Through Deep Learning and Cloud Computing

M. Y. Shakor and M. I. Khaleel, Electronics, vol. 13, no. 24, art. 4860, 2024.

In one line: This is a survey paper. It proves hospital storage is expensive and that deep learning reads scans well, but it never joins the two ideas.

What the authors did: This is a review paper, not an experiment. The authors searched IEEE Xplore, PubMed, Scopus, Web of Science and Google Scholar for work published between 2019 and November

2024. They picked 37 papers and studied them. They also collected results from real hospitals that moved to the cloud, and used all of this to suggest a step-by-step plan for hospitals.

Data they used: There is no experiment and no dataset of their own. The evidence is the 37 papers plus real hospital cases. One example is a group of five hospitals handling 1.2 million scans a year on a hybrid cloud. Another is eight cancer centres sharing 15,000 anonymous cases.

Advantages:

- It gives clear numbers that show why the cloud helps. Time to open a scan dropped from 5 to 7 minutes down to about 30 seconds. Infrastructure cost dropped by about 40 percent. Processing time dropped by about 60 percent.
- It clearly says that storing and moving hospital data is one of the biggest cost problems. This is exactly the problem our project is trying to solve.
- It supports the design choices we need to justify. Hybrid cloud improved rule compliance by about 50 percent. Federated learning kept about 95 percent of the accuracy. Edge computing cut delay by about 40 percent.
- It is honest about failures. 35 percent of hospitals faced data joining problems and 5 percent had a security incident.

Limitations:

- It is only a review. There is no algorithm, no experiment and no code that we can run or repeat.
- Many of the percentages come from hospital reports. The paper does not explain how those numbers were measured, so we cannot check them.
- Storage is discussed only as a general cost problem. The paper never goes down to the level of deciding where one particular scan should be kept.
- Deep learning is used only to read scans and find disease. It is never used to manage the storage system itself.

RESEARCH GAP

This paper proves two facts. First, storing hospital scans costs a lot of money. Second, deep learning is already good enough to read those scans. But the paper keeps these two facts separate. In every study it reviews, deep learning is used only on the doctor's side, to find disease. It is never used on the storage side, to decide where a file should be kept. Nobody takes the result of the image model and sends it back to the storage system. That is the empty space our project fills. The hospital already runs the scan through a model once. That same result can also tell us where the scan should be stored.

How we can improve on this: We will design our system so that one pass of the model does two jobs at the same time — helping the doctor, and deciding the storage tier. We will also measure our project the way this paper shows hospital staff actually judge a system: how fast a scan comes back, how much the monthly bill is, and whether the rules are followed. Accuracy alone is not enough. We will also use the hybrid cloud design this paper recommends, so that patient details stay protected while S3 handles the bulk image storage.

3.3 Paper 3 — Using Compressed JPEG and JPEG2000 Medical Images in Deep Learning: A Review

I. A. Urbaniak, Applied Sciences, vol. 14, no. 22, art. 10524, 2024.

In one line: This paper studies how much a scan can be compressed before deep learning starts making mistakes, but it never links compression to storage tier.

What the authors did: This is a review of what happens to deep learning models when medical images are compressed in a lossy way, which means some detail is thrown away for ever. The author looks only at JPEG and JPEG2000, because these are the only two lossy formats that the DICOM medical standard allows. She compares studies where models were trained and tested on compressed, uncompressed and mixed images.

Data they used: No new experiments were done. The paper collects results from earlier studies. Some used ordinary image sets like MNIST and CIFAR-10. Others used real medical data such as mammograms and digital pathology slides.

Advantages:

- It gives real numbers for the trade-off between saving space and losing accuracy. Pathology images were compressed by 85 percent and the model still kept about 95 percent of its performance.
- It shows where the safe limit ends. For mammograms, accuracy only started dropping after compression crossed about 5000 to 1.
- It gives a useful rule: if a model is trained on images with a mix of different compression levels, it works better on all of them.
- It clearly lists lower storage cost and faster training as benefits, and sticks only to formats that DICOM allows.

Limitations:

- The studies do not agree with each other. Some say accuracy drops only at very high compression. Others say accuracy went up, because the model learned from the compression marks themselves.
- No safe compression limit has been fixed for each scan type or body part. Every result is tied to one particular dataset.
- The author states directly that there is not enough evidence yet to reach a common conclusion.
- There is a risk that a model learns the compression marks instead of the actual disease. It then scores well in training but fails on new images.
- Compression is treated as a single setting applied to the whole dataset. It is not changed from one scan to another.

RESEARCH GAP

In this paper, compression is decided once for the entire archive and never changed. It is also never connected to storage tier. But these are really the same decision seen from two sides. A scan that will almost certainly never be opened again can be compressed heavily to save space. A scan that a doctor is still using must be kept at full quality. No study here changes the compression level for each scan based on how likely that scan is to be needed again, and no study joins the compression choice and the storage class choice into one decision.

How we can improve on this: In our project, compression will be a second output of the same system instead of a fixed setting. If a scan is sent to a hot tier, we keep it at full quality. If it goes to deep archive, we compress it using a ratio that this paper has already shown to be safe for that scan type. This also avoids the problem of the model learning compression marks, because training will still use the full quality copies from the hot tier.

3.4 Paper 4 — Deep learning based medical image compression using cross attention learning and wavelet transform

F. Dai, Scientific Reports, vol. 15, art. 40008, 2025.

In one line: This paper builds a deep learning model that finds the important parts of a scan, but it uses that information only to compress, and then throws it away.

What the authors did: The author builds a compression system with three parts. First, a wavelet transform breaks the image into different levels of detail. Second, a cross attention module decides which parts of the image carry the most information and protects those parts. Third, a small autoencoder cleans up the result before it is finally compressed. A decoder then rebuilds the image.

Data they used: Three public CT datasets were used: LIDC-IDRI, LUNA16 and MosMed. Each was split into 70 percent training, 15 percent validation and 15 percent testing, with 5-fold cross validation. Brain and head MRI scans were also used. Average results were PSNR 40.43 dB, SSIM 0.9715 and 635 milliseconds per image. Training took about 60 to 90 minutes per dataset on an Intel Core i5 CPU.

Advantages:

- It beats JPEG2000, HEVC and BPG, and also three recent deep learning methods, on all quality measures at the same time. So it saves space without losing image quality.
- The attention part looks at the content of the image. It protects the areas that matter medically instead of treating the whole image equally. This is exactly the ability our project needs.
- It was trained on a normal CPU, not an expensive GPU. So it is realistic for a student project or a hospital with a small budget.
- The author clearly says the work is meant for telemedicine and cloud based healthcare storage.

Limitations:

- It was tested mostly on CT and brain MRI. Other scan types are listed as future work.
- It takes about 635 milliseconds per image. This is fine for background work but slow if a hospital is uploading many large studies at once.
- Quality is measured only by mathematical scores such as PSNR and SSIM. No radiologist checked whether the compressed scans are still good enough to make a diagnosis.
- It says nothing about storage. It decides how small to make the file, but not where that file should be kept. There is no cloud cost calculation anywhere in the paper.

RESEARCH GAP

This paper creates exactly the information our project needs, and then throws it away. The attention module already works out which parts of a scan carry important medical detail. But this information is used only to decide how many bits to spend on each area. After compression it is deleted. It is never passed on to any other part of the system. The same compression level is also used for every scan, whether that scan will be opened tomorrow or never again. The paper reduces the size of one image but never looks at the hospital's total bill, which depends on storage class, how long files are kept, and how often they are pulled back.

How we can improve on this: Instead of throwing away the attention result, we will feed it into our tier decision. If a scan gets a high attention score, it has a lot of medical detail, so we keep it at good quality in a faster tier. If the score is low, we can compress it more and move it to archive. We will also test our system in a better way than this paper does: not only PSNR and SSIM, but also whether a disease detection model still works on the compressed scan, and what the monthly S3 bill actually comes to.

3.5 Paper 5 — A Recommendation System Based on AI for Storing Block Data in the Electronic Health Repository

V. Mani, C. Kavitha, S. S. Band, A. Mosavi, P. Hollins and S. Palanisamy, Frontiers in Public Health, vol. 9, art. 831404, 2022.

In one line: This is the closest paper to our idea, but it works on text records instead of images, and it decides only once with no follow up.

What the authors did: The authors build a system that recommends where a block of health data should be stored. They describe the data using four things: how sensitive it is, how large it is, the type of medical care, and the patient's background. They then rate each storage option on five points: storage, cost, security, privacy and performance. Experts fill in a questionnaire, and this is used to train classifiers. They compared Multilayer Perceptron, Random Forest, Random Tree and IB1 using the WEKA tool, along with a deep learning model.

Data they used: The data was not taken from a real hospital system. They created it themselves from a small number of expert ratings, giving two sets of 1,200 and 2,000 records. Random Forest gave the best result, 98.21 percent accuracy on the 1,200 record set. The deep learning model gave 88.70 percent. One point to note is that the conclusion of the paper says the highest accuracy was 96.4 percent, which does not match the 98.21 percent given in the results section.

Advantages:

- This is the closest published work to our project. It shows that choosing a storage location can be treated as a machine learning problem.
- It judges storage options on five different points, not only on cost. This is closer to how a real hospital decides.
- Privacy, security and legal rules are built into the decision from the start.
- It reports precision, recall and F1 score, not only accuracy, so the results can be judged properly.

Limitations:

- The training data comes from expert opinion, not from real usage. So the model learns what experts say they would do, not what actually happened.
- Accuracy dropped on the 2,000 record set because the data was unbalanced. The model also performed poorly on the 'not allowed' class.
- Once a record is labelled, the label is never changed again. So the system cannot adjust when usage patterns change over time.
- The choice is only between three broad options. It never deals with the different tiers inside one cloud provider, which is what our project needs.
- It works on text records and data blocks. Medical images, which take up most of the storage, are never handled, and no image is ever examined.
- It was published in 2022, which is outside the 2023 to 2026 range preferred in the guidelines. I have still included it because it is the closest work to our idea.

RESEARCH GAP

This paper is the closest to our project, and it stops exactly where our work needs to start. The decision is taken only once, when the data first arrives, and it is based on expert opinion. After that the decision is never checked again. If a file turns out to be opened far more often than expected, nothing happens. The system also works on text records, not images, and it only reads details about the data. It never looks at the actual content. In an image archive this means the most useful piece of information available is completely unused.

How we can improve on this: We will keep the good idea of judging storage on several points, but we will make three changes. First, instead of expert opinion we will train on the hospital's real record of which scans were opened again, so the model learns actual behaviour. Second, we will work on one scan at a time instead of a block of text data, and we will add features taken from the image itself. Third, we will check the decision again from time to time using S3 Lifecycle rules, so a scan can be moved up or down as its usage changes. We will also treat a wrong move to archive as a more serious

mistake than paying slightly extra, because a doctor waiting hours for a scan is worse than a small increase in the bill.

4. Overall Research Gap

Putting all five papers together, this is what I found:

- Paper 1 can sort files into storage tiers, but it only reads file details and never looks inside the file.
- Paper 2 proves that hospital storage is costly and that deep learning reads scans well, but it never uses deep learning to manage storage.
- Paper 3 shows how much an image can be compressed safely, but treats compression as one fixed setting and never links it to storage tier.
- Paper 4 builds a model that finds the important parts of a scan, but uses that only for compression and then deletes it.
- Paper 5 does treat storage choice as a learning problem, but works on text records, learns from expert opinion instead of real usage, and decides only once.

So the gap is this. No existing work uses deep learning on the medical image itself, together with the record of how often that scan was opened before, to predict whether it will be needed again and to place it in the correct Amazon S3 storage class. That is what our project will do.

5. Literature Survey Table

Paper	Method	Dataset	Advantages	Limitations	Research Gap
P1 — Khan et al. (2024), Computing, Springer	Gives each file a score from its size, age and how often it is opened, then sends it to one of four tiers: premium, hot, cold or archive. A second version uses game theory so the score weights adjust on their own.	Made-up data built from a Kaggle access log: 871 days, 14,321 files, 2.9 million read requests. Four size versions from 1 TB to 67 TB. Google Cloud prices used.	Four tiers match Amazon S3 well. Simple and fast to run. Works on any cloud. Game theory version was cheaper and faster.	Tested only on made-up data. Leaves out file transfer fees and early delete penalties. Needs files kept 90+ days. Score limits set by hand. Never looks inside the file.	Two scans with the same size, age and view count are treated the same, even if one is medically important. No rule to keep files for the legally required number of years.
P2 — Shakor & Khaleel (2024), Electronics, MDPI	A review of 37 papers published between 2019 and Nov 2024, taken from IEEE Xplore, PubMed, Scopus, Web of Science and Google Scholar, plus real hospital cases, turned into a step-by-step plan for hospitals.	No dataset of their own. 37 papers plus hospital cases, such as five hospitals handling 1.2 million scans a year and eight cancer centres sharing 15,000 cases.	Scan opening time fell from 5-7 min to about 30 sec. Cost fell about 40%. Processing time fell about 60%. Hybrid cloud improved rule compliance by 50%.	Only a review, no experiment or code. Hospital numbers cannot be checked. Storage covered only in general. Deep learning used only to find disease, never to manage storage.	Proves storage is costly and deep learning reads scans well, but never joins the two. The output of the image model is never sent back to the storage system.
P3 — Urbaniak (2024), Applied Sciences, MDPI	A review of what happens to deep learning models when medical images are compressed in a lossy way, covering only JPEG and JPEG2000 because those are the two formats DICOM allows.	No new experiments. Collects results from earlier studies using MNIST, CIFAR-10, mammograms and digital pathology slides.	Gives real numbers: pathology images compressed 85% still kept about 95% of performance; mammogram accuracy dropped only past 5000:1. Mixed compression levels in training help.	Studies disagree with each other. No safe limit fixed per scan type or body part. Author says evidence is not enough yet. Models may learn compression marks instead of disease.	Compression is one fixed setting for the whole archive. It is never changed per scan and never connected to which storage tier the scan goes into.
P4 — Dai (2025), Scientific Reports, Nature	Wavelet transform splits the image into detail levels, a cross attention module protects the important areas, a small autoencoder cleans it up, then the image is compressed and rebuilt.	LIDC-IDRI, LUNA16 and MosMed CT datasets, split 70/15/15 with 5-fold cross validation, plus brain and head MRI. Avg PSNR 40.43 dB, SSIM 0.9715, 635 ms per image. Trained on an i5 CPU.	Beats JPEG2000, HEVC, BPG and three recent deep learning methods on all quality measures at once. Attention protects medically important areas. Runs on a normal CPU.	Mostly CT and brain MRI only. 635 ms per image is slow for bulk upload. Judged only by maths scores, no doctor checked the images. Says nothing about storage or cost.	The attention module finds which parts of a scan matter, then deletes that information after compressing. It is never reused. Same compression for every scan. No total cost view.
P5 — Mani et al. (2022), Frontiers in Public Health	Describes health data by sensitivity, size, care type and patient background, rates storage options on storage, cost, security, privacy and performance, then trains MLP, Random Forest, Random Tree and IB1 in WEKA plus a deep learning model.	Made-up sets of 1,200 and 2,000 records built from a small number of expert ratings. Random Forest best at 98.21% on 1,200 records; the conclusion says 96.4%. Deep learning model 88.70%.	Closest work to our project. Shows storage choice can be a machine learning problem. Judges storage on five points, not just cost. Privacy and legal rules built in.	Trained on expert opinion, not real usage. Accuracy dropped on the unbalanced 2,000 record set. Labels never updated. Only three broad options, not tiers. Text records only, no images.	Decides only once when data arrives and never checks again. Works on text blocks, not scans. Reads only data details, never the actual image content.